Individual Assignment

Raspberry Pi Distance Monitoring with HC-SR04, OLED and Web Interface

1. Context

This individual assignment aims to develop practical skills in embedded systems prototyping using a Raspberry Pi, an ultrasonic distance sensor, a local display, and a web interface.

The system shall be developed in **Python** and shall measure distance using an **HC-SR04 ultrasonic sensor** connected to the Raspberry Pi.

The measured distance shall be presented simultaneously:

- on a **0.96-inch 128 × 64 I2C OLED display**;
- on a **web page hosted by the Raspberry Pi**.

Students are allowed to use Artificial Intelligence tools during the development of the assignment. These may include AI tools integrated into Visual Studio Code, such as **GitHub Copilot Chat**, as well as other generative AI systems.

The use of AI is permitted and may be part of the development process. However, the student remains fully responsible for the submitted work and must be able to **understand, explain, validate, debug, and modify the implemented solution**.

The assessment will therefore focus not only on whether the prototype operates correctly, but also on the student's understanding of the hardware, software architecture, sensor operation, timing measurements, I2C communication, web interface, and design decisions.

2. Objectives

The main objective is to design and implement an embedded distance-monitoring system based on a Raspberry Pi.

At the end of the assignment, the student should be able to:

- interface an HC-SR04 ultrasonic sensor with a Raspberry Pi;
- generate and measure digital timing signals using GPIO;
- calculate distance from ultrasonic pulse travel time;
- understand the electrical compatibility between the sensor and Raspberry Pi GPIO;
- communicate with an I2C peripheral;
- display information on an OLED display;
- implement a basic web application in Python;
- present sensor data through a web browser;
- organize software into appropriate functional components;

- manage periodic acquisition and display updates;
- detect and handle sensor and communication errors;
- critically use AI-assisted programming tools;
- explain and defend the implemented solution.

3. Required Hardware and Software

The prototype shall include at least:

- Raspberry Pi;
- HC-SR04 ultrasonic distance sensor (The available HC-SR04 sensor should work with 3.3V power-supply);
- 0.96-inch OLED display, 128 × 64 pixels, with I2C interface;
- breadboard and connection wires;
- network connection;
- Python 3 development environment.

A suitable Raspberry Pi operating system and the required Python packages may be used.

4. Hardware Considerations

The available HC-SR04 sensor should work with 3.3V power-supply, and uses two digital signals:

- **TRIG** – input used to initiate a measurement;
- **ECHO** – output pulse whose duration is related to the measured distance.

The Raspberry Pi GPIO operates at **3.3 V logic levels**, therefore, no adaptation circuits are needed.

5. Principle of Operation of the HC-SR04

The student shall understand the operating principle of the HC-SR04.

A measurement is initiated by applying a short pulse to the TRIG input.

The sensor emits an ultrasonic burst and produces a pulse on the ECHO output.

The duration of the ECHO pulse is related to the time required for the ultrasonic wave to:

1. travel from the sensor to the object;
2. reflect from the object;
3. return to the sensor.

The distance shall be calculated from the measured time-of-flight.

A typical relationship is:

[

d = \frac{v \times t}{2}

]

where:

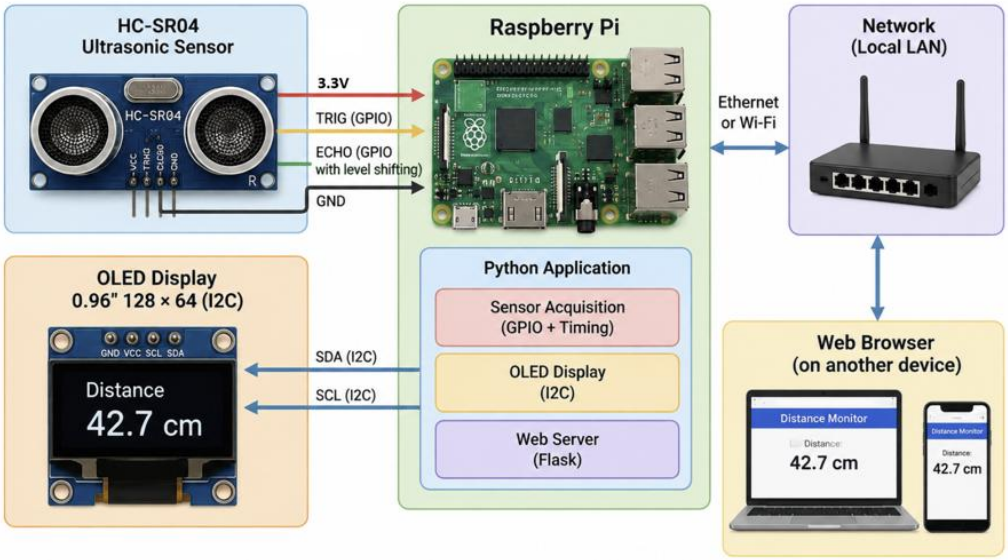
- (d) is the distance;
- (v) is the speed of sound;
- (t) is the measured round-trip time.

The factor of 2 is required because the ultrasonic signal travels to the object and back to the sensor.

The student shall be able to explain this relationship during the individual discussion.

6. System Architecture

The minimum expected architecture is:



The same measured distance shall be available both locally on the OLED display and remotely through the web interface.

7. Minimum Functional Requirements

The developed system shall:

1. initialize the Raspberry Pi GPIO interface;
2. configure the HC-SR04 TRIG and ECHO pins;
3. generate the trigger pulse required to initiate a measurement;
4. measure the duration of the ECHO pulse;
5. calculate the corresponding distance;

6. verify whether the measurement is valid;
7. initialize and communicate with the 128 × 64 I2C OLED display;
8. display the current distance on the OLED;
9. run a web server on the Raspberry Pi;
10. provide a web page accessible from another device on the same network;
11. display the current distance on the web page;
12. periodically update the measured distance;
13. periodically update the OLED information;
14. allow the web page to obtain updated measurement data;
15. provide relevant diagnostic messages through the terminal or system log;
16. handle sensor measurement failures and timeouts;
17. operate continuously without requiring manual intervention.

The acquisition period and display/web update periods shall be clearly defined and justified.

8. HC-SR04 Measurement Implementation

The distance measurement shall be implemented in Python.

The student shall understand and be able to explain:

- how the TRIG pulse is generated;
- how the ECHO pulse is detected;
- how pulse duration is measured;
- how the measured time is converted into distance;
- why the propagation time must be divided by two;
- how the speed of sound affects the result;
- how measurement timeouts are implemented;
- how invalid measurements are detected.

Dedicated libraries that directly return the distance value may be used only if explicitly permitted by the lecturer.

Implementations that directly control the GPIO signals and perform the timing measurement in the student's code are preferred, since they demonstrate a better understanding of the sensor operating principle.

9. Software Architecture

The Python application shall be organized into clear functional components.

The implementation should avoid placing all functionality in a single procedural block.

The student is encouraged to create functions, classes, or modules responsible for tasks such as:

- GPIO initialization;
- ultrasonic measurement;
- distance calculation;
- measurement validation;
- OLED initialization;
- OLED update;
- web-server initialization;
- web-page generation;
- storage of the most recent valid measurement;
- error handling.

The student shall be able to explain the execution flow of the application and the interaction between these components.

10. OLED Display

The system shall use a **0.96-inch 128 × 64 OLED display with I2C interface**.

The OLED shall display at least:

- the measured distance;
- the corresponding unit.

For example:

Distance = 42.7 cm

Additional information may also be displayed, such as:

- measurement status;
- minimum or maximum distance;
- acquisition time;
- Raspberry Pi IP address;
- system status.

The student shall understand and be able to explain:

- the I2C communication principle;
- the OLED I2C address;
- the Raspberry Pi I2C bus;

- the purpose of SDA and SCL;
- how the display is initialized;
- how information is written to the display.

The use of an OLED graphics library is permitted.

11. Web Interface

The Raspberry Pi shall host a web application that presents the measured distance.

A lightweight Python web framework such as **Flask** may be used.

The web page shall, as a minimum:

- show the current distance;
- show the corresponding unit;
- indicate that the measurement originates from the Raspberry Pi;
- provide updated measurement information.

The student may implement the web-page update mechanism using, for example:

- periodic page refresh;
- JavaScript polling;
- HTTP requests to a JSON endpoint;
- another technically justified solution.

The browser shall not communicate directly with the HC-SR04 sensor.

The Python application shall maintain the latest valid sensor measurement, which shall then be made available to the web interface.

12. Data Endpoint

In addition to the main web page, the application should provide a simple HTTP endpoint that returns the most recent sensor measurement in a machine-readable format.

For example:

/api/data

may return:

```
{
  "distance_cm": 42.7
}
```

The student shall understand and be able to explain:

- what an HTTP endpoint is;

- what JSON is;
- how the web page can request data from the endpoint;
- how the Python application provides the latest sensor measurement.

13. Error Handling and Robustness

The implementation shall consider possible failure conditions.

At minimum, the system should handle situations such as:

- no ECHO pulse detected;
- ECHO pulse remaining active for an excessive time;
- distance outside an expected operating range;
- invalid measurement;
- OLED not detected;
- I2C communication failure;
- web-server initialization failure;
- unexpected Python exception.

The software shall include appropriate **timeouts** to prevent the program from becoming permanently blocked while waiting for the ECHO signal.

Whenever possible, errors should be reported through the terminal, OLED, web interface, or an appropriate combination of these.

14. Use of Artificial Intelligence Tools

Artificial Intelligence tools may be used during the development of this assignment.

Examples include:

- GitHub Copilot;
- GitHub Copilot Chat;
- ChatGPT;
- other code-generation or AI-assisted development tools.

AI may be used for activities such as:

- generating initial code;
- suggesting software architecture;
- creating HTML or CSS;
- debugging;
- explaining Python libraries;

- identifying programming errors;
- generating documentation;
- suggesting alternative implementations.

However, the student remains responsible for the complete submitted solution.

The student must understand all relevant submitted code, including code generated or modified using AI tools.

The inability to explain significant parts of the implementation during the individual discussion may significantly affect the final assessment, even if the system operates correctly.

15. Documentation of AI Use

The technical report shall contain a section entitled:

Use of Artificial Intelligence Tools

This section shall briefly identify:

- which AI tools were used;
- for which development tasks they were used;
- examples of relevant prompts or interactions;
- which parts of the implementation were substantially generated or modified by AI;
- errors, limitations, or inappropriate suggestions produced by the AI;
- how the student verified the technical correctness of the generated solution.

The purpose is not to penalize the use of AI, but to evaluate the student's ability to use these tools critically and responsibly.

16. Technical Report

Each student shall submit an individual technical report.

The report should be concise and technically oriented and shall include at least the following sections.

16.1 Introduction

Brief description of the objective of the prototype and its possible applications.

16.2 System Architecture

Description of the complete system.

A block diagram shall show:

- HC-SR04;
- Raspberry Pi;

- OLED display;
- web server;
- browser.

The main interfaces and data flow shall be identified.

16.3 Hardware

Description of:

- Raspberry Pi model;
- HC-SR04 sensor;
- OLED display;
- electrical connections;
- power supply;

A wiring diagram or schematic shall be included.

16.4 HC-SR04 Operating Principle

The report shall explain:

- ultrasonic distance measurement;
- TRIG signal;
- ECHO signal;
- time-of-flight measurement;
- distance calculation;
- speed of sound;
- measurement range and limitations.

16.5 GPIO and Timing

The report shall describe:

- GPIO pins used;
- input/output configuration;
- trigger-pulse generation;
- ECHO timing measurement;
- timeout mechanism.

16.6 I2C and OLED Communication

The report shall describe:

- I2C configuration;
- OLED address;

- Raspberry Pi pins used;
- initialization procedure;
- method used to display information.

16.7 Software Architecture

Description of the main Python components and program execution flow.

A flowchart, state diagram, UML diagram, or equivalent representation is recommended.

16.8 Web Application

Description of:

- web framework used;
- server operation;
- network port;
- implemented routes;
- JSON endpoint, if implemented;
- communication between the sensor-acquisition code and the web interface;
- web-page update mechanism.

16.9 Results

Evidence of correct system operation shall include:

- photograph of the complete prototype;
- OLED showing a measured distance;
- screenshot of the web page;
- examples of measurements at different known distances;
- comparison between measured and expected distances;
- observations regarding measurement accuracy and stability.

16.10 Error Handling and Robustness

The student shall describe how the implementation reacts to relevant failure conditions.

16.11 Use of Artificial Intelligence Tools

As described above.

16.12 Conclusions

The student shall critically discuss:

- achieved results;
- measurement accuracy;
- limitations;

- possible improvements;
- suitability of the implemented prototype for real applications.

17. Experimental Validation

The system shall be experimentally evaluated.

The student shall perform measurements for several known distances and compare them with the values reported by the HC-SR04.

For example, measurements may be performed approximately at:

- 10 cm;
- 20 cm;
- 50 cm;
- 100 cm.

The exact test distances may be selected by the student.

The report should contain a table similar to:

Reference Distance	**Measured Distance**	**Error**
10 cm	...	...
20 cm	...	...
50 cm	...	...
100 cm	...	...

The student shall comment on measurement errors and possible causes.

18. Individual Presentation and Live Demonstration

Each student shall individually present the developed prototype to the lecturer.

The presentation shall include a **live demonstration** of the complete system.

The student shall demonstrate:

- the physical Raspberry Pi prototype;
- the HC-SR04 sensor;
- distance measurement;
- information presented on the OLED;
- access to the web page from another device;
- updated distance information on the web page;

- relevant terminal or diagnostic output.

The student may also be asked to place an object at different distances from the sensor and demonstrate that both the OLED and web page respond correctly.

19. Individual Technical Discussion

Following the demonstration, each student will participate in an individual technical discussion with the lecturer.

This discussion is an essential component of the assessment.

Students may be asked to explain topics such as:

- how the HC-SR04 measures distance;
- the function of TRIG;
- the function of ECHO;
- how the ECHO pulse duration is measured;
- why the distance equation contains a division by two;
- why the ECHO signal cannot normally be connected directly to a Raspberry Pi GPIO;
- how the voltage divider works;
- how Python accesses GPIO;
- why a timeout is necessary;
- what happens if no object is detected;
- how I2C works;
- how the OLED is addressed;
- how the Python web server operates;
- what happens when the browser requests a web page;
- what an HTTP route is;
- what JSON is;
- how the most recent distance value is shared with the web interface;
- which operations are blocking;
- how AI tools were used during development.

The lecturer may ask the student to explain specific portions of the submitted source code.

20. Live Code Modification

During the individual assessment, the lecturer may request one or more small modifications to the developed system.

Examples include:

- changing the acquisition period;
- changing the OLED update period;
- changing the displayed distance unit from centimetres to millimetres;
- adding the distance in metres;
- adding a minimum-distance alarm;
- displaying an alert when an object is closer than a specified threshold;
- changing the OLED layout;
- adding the Raspberry Pi IP address to the display;
- adding a new JSON field;
- changing the web-page update rate;
- displaying the minimum or maximum measured distance;
- calculating the average of the last N measurements;
- changing the distance threshold during the demonstration;
- modifying the web page to display measurement status.

The student shall modify, execute, and test the application.

AI tools may be used during this phase if permitted by the lecturer. However, the student must understand and explain the requested change and the resulting implementation.

21. Questions During the Assessment

The student should be prepared to answer questions such as:

- Why is the HC-SR04 powered from 5 V?
- What voltage is accepted by the Raspberry Pi GPIO?
- Why is a voltage divider required on the ECHO line?
- What is the purpose of the TRIG signal?
- What does the ECHO pulse represent?
- How is pulse duration converted into distance?
- Why is the propagation time divided by two?
- What value for the speed of sound is used?
- Would temperature affect the measurement?
- What happens if the ECHO signal never changes state?
- Why should a timeout be implemented?

- What is the maximum useful measurement rate?
- Why can individual measurements fluctuate?
- How could measurement stability be improved?
- What is I2C?
- What are SDA and SCL?
- What is the OLED I2C address?
- How can connected I2C devices be detected?
- What is an HTTP request?
- What happens when a browser accesses the Raspberry Pi IP address?
- What is an HTTP endpoint?
- What is JSON?
- How does the web page obtain the latest distance measurement?
- Does the browser communicate directly with the sensor?
- Which parts of the program were generated or suggested by AI?
- How did you verify that the AI-generated code was correct?
- What would you improve for a permanent or industrial installation?

The exact questions may vary between students.

22. Assessment Criteria

The assessment shall consider the following components:

Component	**Weight**
Functional prototype and system integration	15%
Technical report and experimental validation	20%
Software implementation and technical quality	20%
Individual presentation and live demonstration	15%
Individual technical discussion	20%
Live code modification / problem-solving task	10%

The **Software implementation and technical quality** component shall consider:

- correct GPIO configuration;
- correct implementation of the HC-SR04 timing sequence;

- correct distance calculation;
- appropriate timeout implementation;
- software organization;
- OLED integration;
- web-server implementation;
- error handling;
- robustness;
- readability and maintainability;
- appropriate use of software libraries.

23. Important Assessment Principle

A fully functional application does not, by itself, demonstrate achievement of the learning objectives.

Modern AI tools can generate Python code, Flask applications, HTML pages, sensor-interface examples, and OLED control code with minimal user intervention.

For this reason, particular importance will be given to the student's ability to:

- explain the physical measurement principle;
- explain the electrical interface;
- interpret the source code;
- understand GPIO timing;
- understand I2C communication;
- understand the web application;
- identify the role of each library;
- diagnose problems;
- justify implementation choices;
- modify the application;
- predict the system behavior;
- critically evaluate AI-generated solutions.

The submitted source code is therefore considered both an implementation and a technical object that the student must be able to analyse and defend.

24. Deliverables

Each student shall submit:

1. the **technical report in PDF format**;
2. the complete **Python source code**;
3. HTML, CSS, JavaScript, templates, or other files required by the web application;
4. any configuration files required to execute the project;
5. the system architecture diagram;
6. the complete hardware wiring diagram;
7. experimental validation results;
8. evidence of OLED and web-interface operation;
9. a short record or description of the relevant AI-assisted development process.

The student must also attend the scheduled **individual presentation, live demonstration, and technical discussion**.

25. Individual Work

This is an **individual assignment**.

Discussion of general technical concepts between students is permitted. However, each student must develop, understand, submit, present, and defend their own implementation.

The student is individually responsible for all material submitted under their name, regardless of whether conventional documentation, online examples, software libraries, or AI-assisted development tools were used.

26. Submission Deadline

The assignment shall be submitted no later than three weeks after the assignment is released (November, 6, 2026).

The assignment shall be submitted by the date specified by the lecturer.

The individual presentations, live demonstrations, and technical discussions will take place after the submission deadline, according to a schedule defined by the lecturer.

Late submissions will have the grade reduced with 0,5 points per day.